How Airbnb Safeguards Changes in Production

medium.com/airbnb-engineering/how-airbnb-safeguards-changes-in-production-9fc9024f3446

Michael Lin July 11, 2022

Creative engineers and data scientists building a world where you can belong anywhere.
http://airbnb.io

Part I: Evolution of Airbnb's experimentation platform

Press enter or click to view image in full size

Introduction

As Airbnb has grown to a company with over 1,200 developers, the number of platforms and channels for pushing changes to our product — and the number of daily changes we push into production — has also grown tremendously. In the face of this growth, we constantly need to scale our ability to detect errors before they reach production. However, errors inevitably slip past pre-production validation, so we also invest heavily in mechanisms to detect errors quickly when they do make it to production. In this blog post we will cover the motivations and foundations for a system for safeguarding changes in production, which we call Safe Deploys. Two following posts will cover the technical architecture in detail for how we applied this to traditional A/B tests, and code deploys respectively.

Continuous Delivery and Beyond

Airbnb's continuous delivery team recently wrote about our adoption of Spinnaker, a modern CI/CD orchestrator. Spinnaker supports Automated Canary Analysis (ACA) during deployment, splitting microservice traffic by request to compare versions of code to see if performance, error rates, or other key metrics are negatively impacted. If metrics for the new version regress, Spinnaker automatically rolls back the deployment, significantly reducing the time to remediate a bad push.

ACA at Airbnb has indeed caught a large number of errors early in the deployment process. However, it has a number of limitations:

- Spinnaker's ACA tests against changes to microservices. However, microservice updates are not the only source of errors that can be pushed into production. For instance, Android and iOS apps follow a release process through their respective app stores. Many "production pushes" at Airbnb may involve no new code at all, and are strictly applied through configuration changes. These changes include marketing campaigns or website content created with Airbnb's internal content management systems. While seemingly benign, pushes through these systems can have dramatic effects. For example an incident was once caused when a marketing campaign was mistakenly applied to all countries except one, instead of the original intent of targeting one specific country. This simple mistake led to empty search results for nearly all users globally, and required over an hour to identify and revert.
- Spinnaker's ACA is driven by local system metrics, such as a microservice's local performance and error rates; not end-to-end business metrics, such as search click-through rates and booking rates. While roll-backs based on local system metrics are valuable, they aren't sufficient, as some of our most costly bugs impact end-to-end business metrics but not local system metrics. For instance in 2020, a simple frontend change was deployed to production without being tested on a specific browser that did not support the CSS used, preventing users on that browser from booking trips. This had no impact on system metrics, but directly impacted business metrics. Unfortunately, adding business metrics to Spinnaker's ACA system is not possible because Spinnaker randomizes traffic by request, therefore the same user may be exposed to multiple variants. Business metrics, however, are generally user based and require each user to have a fixed variant assignment. More fundamentally, it's not possible because business metrics need to be measured end-to-end and when two microservices undergo ACA at the same time, Spinnaker has no way of distinguishing the respective impact of those two services on end-to-end business metrics.
- Spinnaker's ACA tests at the level of the entire microservice. However, it's often the case that two features are being worked on at the same time within a microservice. When ACA fails, it can be hard to tell which feature caused the failure.

While we heavily depend upon Spinnaker's ACA at Airbnb, it became clear there was an opportunity to complement it and address the above limitations where the circumstances call for it.

Experimentation Reporting Framework (ERF)

A/B testing has long been a fixture in product development at Airbnb. While sharing some qualities with ACA in counterfactual analysis, A/B testing has focused on determining whether a new feature improves business outcomes, versus determining whether that feature causes a system regression. Over the years Airbnb has developed our Experimentation Reporting Framework (ERF) to run hundreds of concurrent A/B experiments across a half dozen platforms to determine whether a new feature will have a positive impact.

ERF addresses the limitations of ACA listed above:

- With each new platform, an ERF client has been introduced to support A/B testing on it. This includes mobile, web, and backend microservices. APIs were also introduced to provide config systems an avenue to treat config changes as A/B tests.
- ERF is driven *primarily* by end-to-end business metrics. On the technical side, it randomizes by user, not request, and it is able to distinguish the impact of hundreds of experiments running concurrently. ERF taps into Airbnb's central metrics system to access the thousands of business metrics and dimensions Product and Business teams have defined to measure what matters most to Airbnb overall.
- Where Spinnaker's ACA runs its experiments at the level of an entire microservice, ERF runs its experiments based on what are basically feature flags embedded into the code. Thus, if multiple features are being developed concurrently in the same microservice, ERF can determine which one is impacting the business metrics .

The above characteristics of ERF address the limitations of ACA, but ERF also had a limitation of its own: it was a daily-batch system generating interactive reports intended to be consumed by human decision makers. To address the limitation of Spinnaker's ACA, ERF needed to evolve into a near real-time system that can directly control the deployment process without human intervention.

Press enter or click to view image in full size

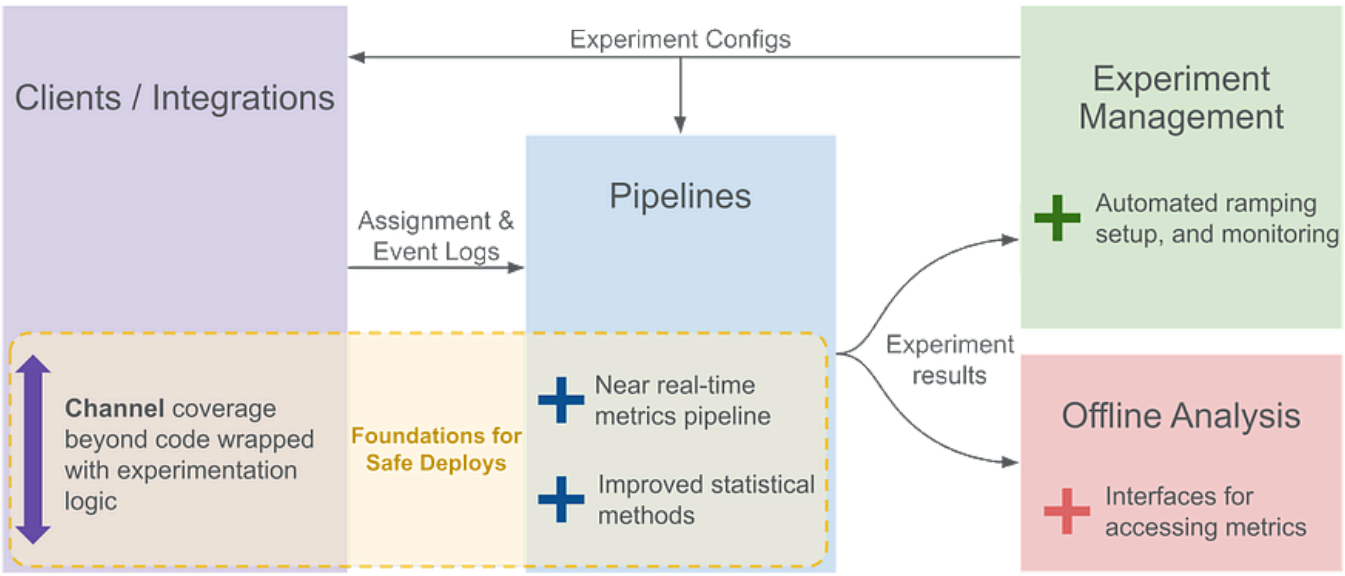

Figure 1: Areas of the ERF Platform augmented to support near real-time experimentation

This evolution had implications on both the data science behind ERF, and its software architecture. We describe the former in this post, and will describe the latter in the next post of this series.

Realtime ERF — The Data Science

The foundation of solid data science is solid data engineering. On the data engineering side, we needed to revisit the definitions of the business metrics to be computed in real-time. The metrics computed by the batch ERF system were designed for accuracy, and could take advantage of complex joins and pre-processing to achieve this. Near real-time metrics did not have this luxury, and required simplification to meet low latency requirements.

Join Medium for free to get updates from this writer.

Remember me for faster sign in

Not only did we have to build new metrics, but we knew we would have to build new statistical tests as well. It is imperative for safe deployment systems to not be noisy, otherwise people will stop using it. Traditional methods like T-Test suffer from a variety of issues that would be extremely problematic when implemented in a real-time system. Two issues in particular are false positives due to (1) peeking (looking before a predetermined amount of time) and (2) heavily skewed data.

When monitoring whether or not a metric has changed in real-time, users want to be notified as soon as the model has the confidence that this is true. However, doing so naively results in the first issue, peeking. In traditional A/B testing, the statistical test is only applied once after a predetermined time, because there is a chance that a significant result is due to randomness and not an actual effect. For real-time ERF, we aren't making just one test, since, depending on how long we wait to take the test, we're at risk for either taking too long to detect some errors, or missing other errors that take longer to surface. Instead, we want to check (peek at) the model every 5 minutes so that we can react quickly. With a p-value of 0.05 running 100 A/A comparisons, one could expect to have ~5 significant results that are actually false positives. We can transfer this issue to computing p-values on the same data set multiple times. Each evaluation results in a 5% chance of a false positive and so over multiple evaluations, the chance of having 1 or more false positives approaches 100%.

Press enter or click to view image in full size

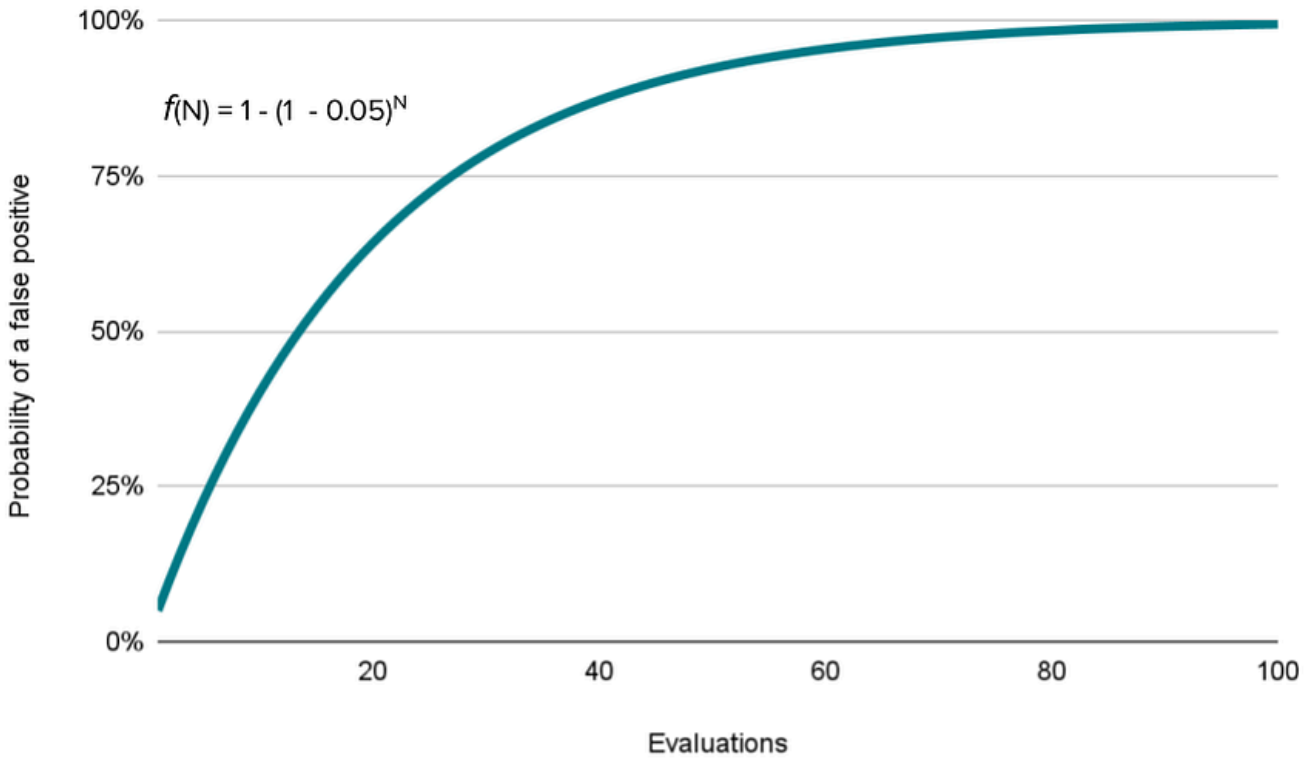

Figure 2: Increasing evaluations inevitably lead to false positives

To balance early detection without noisiness, we utilize sequential analysis . Sequential methods do not assume a fixed sample size (i.e., checking the model once) and allow us to continually monitor a metric without worrying about false positives incurred due to peeking. One way to correct for false positives (Type 1 Errors) is by applying a Bonferroni correction. If you check your model for statistical significance four times and want to guarantee a 5% overall false positive rate, you need to divide your p-value by four, meaning only results with p-value at or under 1.25% are valid. However, doing so is too conservative since each check is dependent. They are dependent because each check has the same base of data only adding additional observations as time goes on. Sequential models take this dependence into account while guaranteeing false positives rates more efficiently than Bonferroni. We use two different sequential models, SSRM (Sequential Sample Ratio Mismatch) for count metrics, and Sequential Quantiles (Howard, Ramdas) for quantile metrics.

The second issue that we needed to solve in order to be robust is handling skewed data. Performance metrics like latency can have extremely heavy tails. Models that assume a normal distribution won't be effective because the Central Limit Theorem does not come into effect. By applying Sequential Quantiles, we can ignore assumptions about the metric's distribution and directly measure the difference between arbitrary quantiles.

Press enter or click to view image in full size

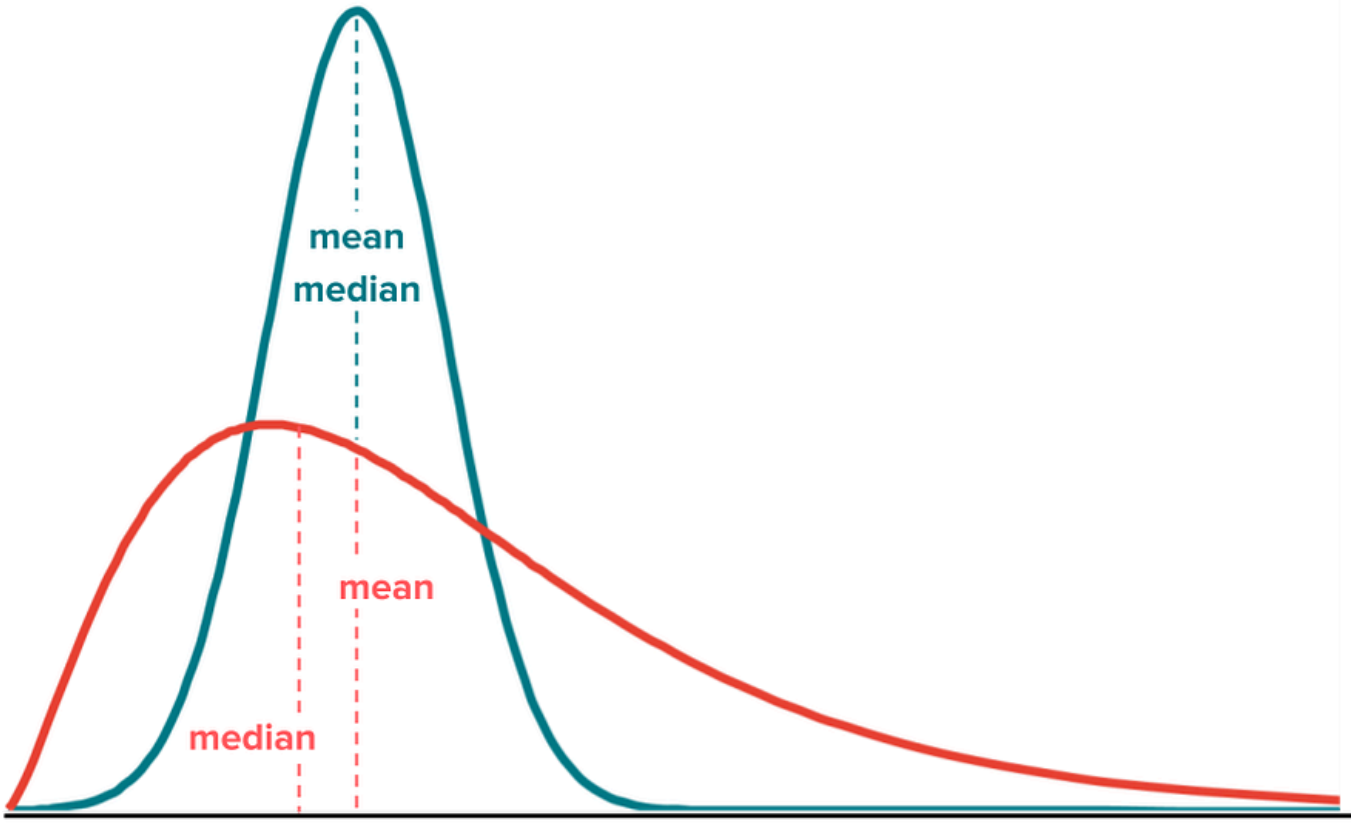

Figure 3: Metrics may have non-normal distributions

Lastly, many important measures are not independent. Metrics like latency and impressions have within-user correlation, so each event in the data cannot be treated as an independent unit. In order to counteract skew, we aggregate all measures into user metrics first before evaluating statistical models.

Conclusion

With the statistical methods in place to evaluate business metrics in near real-time, we could now detect problems that were invisible to Spinnaker, or required too much lead time to rely on traditional ERF experiments.

Press enter or click to view image in full size

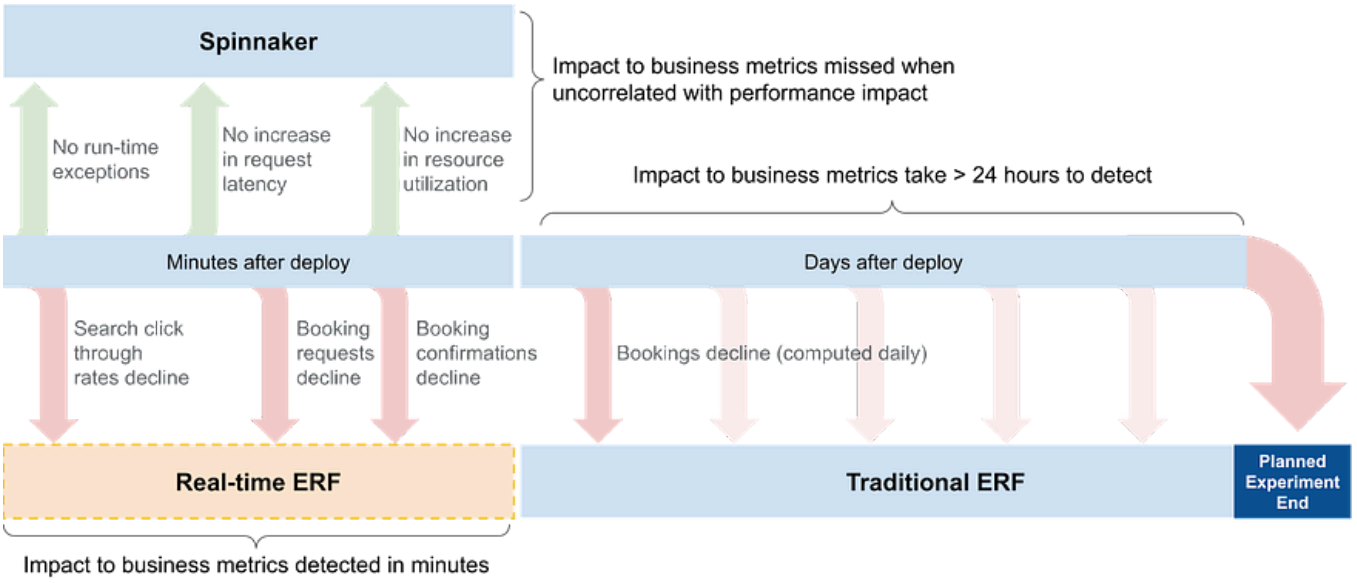

Figure 4: How Real-time ERF fits between Spinnaker and Traditional ERF

Operationalizing the newly created near real-time metrics and statistical methods required further engineering, but more challenging, it required changing the experimentation culture at Airbnb. In the following post we will detail how our near real-time metrics pipeline was built, how these metrics powered automated decision making, and how we drove adoption across the company.

Interested in working at Airbnb? Check out these open roles:

Senior Software Engineer, Metric Infrastructure

Staff Software Engineer, Real-time Stream Processing Platform

Staff Software Engineer — ML Ops Platform

Staff Software Engineer, Cloud Infrastructure

Appreciations

Thanks to Adrian Kuhn, Alex Deng, Antoine Creux, Erik Iverson, George Li, Krishna Bhupatiraju, Preeti Ramasamy, Raymie Stata, Reid Andersen, Ronny Kohavi, Shao Xie, Tatiana Xifara, Vincent Chan, Xin Tu and the OMNI team.